

vivo-linkkit

Building a Linux client for vivo Office Kit — from zero to a working tool.

CLEAN-ROOM

INTEROPERABILITY

ARCH LINUX

REVERSE ENGINEERING

WHAT THIS IS

An independent, clean-room interoperability client. We do **not** port or modify vivo's software. We observe how the phone and the official desktop client talk to each other, write that protocol down, and reimplement a client that speaks it — the same approach that produced **scrcpy** (Android mirroring without a vendor blob) and **libimobiledevice** (Apple's device stack, reimplemented from scratch).

The one question that decides everything

Before investing a month, Phase 1 must answer **how pairing is authenticated**. There are three possible worlds, and they have very different outcomes:

Auth model	What it means	Verdict
Local trust	ECDH key exchange + on-screen confirmation, pinned key stored both sides (how KDE Connect and Apple's lockdown work)	Very feasible, fully distributable
User-account login	Each user signs into their own vivo account; no shared secret to redistribute	Feasible — drive the user's login
Vendor credential	The desktop client holds a vivo-signed certificate/token	Only viable if login is user-account-based; otherwise not distributable

DO THIS FIRST

Find the pairing model in Phase 1. Everything downstream depends on it. It is far cheaper to learn "this needs a vendor credential" in week 1 than in week 4.

Two ways in — use both, in this order

1. The phone side, first

Android apps decompile to near-readable Java. Running `jadx` on the Office Kit phone agent typically surfaces port numbers, JSON/protobuf field names, the discovery broadcast format, the crypto suite, and often the entire state machine as method names. This is roughly 10× the signal of staring at x86 disassembly.

2. The Windows client, second

You can't easily instrument the phone (vivo has effectively stopped issuing bootloader unlocks → no root → no `frida-server` on a platform-signed system app). But you have **total control of a Windows VM**. Frida on Windows needs no privilege gymnastics: hook the send/recv calls and the crypto functions and you read the **plaintext** protocol without ever breaking the encryption. Both endpoints implement the same protocol, so instrumenting either one is equivalent.

FIRST THING TO CHECK ON THE BINARY

Run `strings` on the client and inspect its DLL imports. If it's **Electron**, `app.asar` unpacks and may hand you the protocol in readable JavaScript. If it's **Qt/QML**, the QML resources are similarly extractable. Either saves you weeks.

The instrumentation rig

- **Windows 11 under QEMU/KVM**, USB controller passed through, **bridged** networking — not NAT. Discovery is almost certainly UDP broadcast or mDNS and needs the phone and client on the same layer-2 segment.
- `usbmon` + Wireshark on the host for the USB transport.
- Wireshark on the bridge for the wireless transport.
- `mitmproxy` for any cloud calls during login.
- Frida in the guest for plaintext.

Do USB before Wi-Fi. The wireless path likely stacks BLE proximity advertising plus Wi-Fi Direct/P2P negotiation on top of the core protocol — a second protocol to reverse. USB is a clean single pipe; solve it first.

Phase plan

P0 Recon & setup — days

Goal: real data on the wire, not guesses.

- `scripts/bootstrap.sh` — install host deps, create the venv.
- `recon/01_inventory.sh` — device identity + candidate packages + open ports.
- `recon/02_decompile.sh <pkg>` — pull & decompile the phone agent, auto-grep for protocol signals.
- `recon/03_capture.sh` — usbmon / network / port-sweep captures.

P1 Protocol map — 1-2 weeks

Goal: answer the pairing question; fill `protocol/PROTOCOL.md`.

- Identify discovery mechanism, ports, and packet formats.
- Determine the auth model (local trust / account / vendor credential).
- Document framing: magic bytes, header layout, endianness, encryption boundary.
- Publish `PROTOCOL.md` even at 30% — it recruits collaborators.

P2 Exploration client — 2-4 weeks

Milestone that proves viability: pair, then write **one decodable H.264 frame** to disk. Verify with `ffmpeg -i first_frame.h264 -frames:v 1 out.png`.

- Python on purpose — you'll rewrite the parser many times; Python makes that minutes.
- `src/vivolinkkit/explore.py`: `discover` → `pair` → `grab_one_frame`.
- If the frame decodes, the rest is grinding, not research.

P3 Real client — 1-2 months

Goal: smooth mirroring + input control.

- Port stabilized parsers from Python to Rust or Go.
- Feed the video stream into ffmpeg/libav or GStreamer; render in a window.
- Send input events (touch/key/mouse) back up the channel.

P4 Services — ongoing

Goal: feature parity with the pieces people actually use.

- Two-way file transfer, clipboard sync, notification mirroring.

P5 Packaging & breadth — ongoing

Goal: other people can install it in one command.

- Write a PKGBUILD; publish to the AUR.
- Only now broaden beyond your one device family.

Two decisions that multiply your odds

1. **Publish the protocol before the client works.** A repo with a `PROTOCOL.md` that documents discovery and pairing — even 30% complete — is what attracts the three other people who own vivo phones and know libavcodec. Community reverse-engineering of vendor mirroring stacks moves fastest once the handshake is written down publicly.

2. **Scope to one device family first.** Your exact model, your exact OriginOS version, USB only. A working narrow thing beats a broken general thing. Generality comes after the first success, not before.

Rules that keep the project clean & legal

- Never commit vivo's binaries, APKs, DLLs, or extracted keys/certs — they live in the git-ignored `captures/`.
- Clean-room discipline: implement from the documented protocol, not by transliterating decompiled code. Decompiled code informs the spec; the spec informs the source.
- Never ship vendor secrets. If pairing needs a vivo credential, drive the user's own account login instead of embedding one.

LEGAL FOOTING (NOT LEGAL ADVICE)

Reverse engineering strictly to obtain information necessary for interoperability of an independently created program is expressly permitted in several jurisdictions — for example India's Copyright Act §52(1)(ab), with analogous provisions elsewhere. Keep it clean: no redistributed binaries, no shipped keys, implementation written from the documented protocol, and an interoperability statement in the README.

Companion to the vivo-linkkit repository. Start with `CLAUDE.md` at the repo root, run `scripts/bootstrap.sh`, then `recon/01_inventory.sh`. This roadmap is a living plan — revise as recon replaces assumptions with facts.